

Initialization Suggestions for `misclassCRC`

Internal technical note

24 August 2026

1 Purpose

This note proposes an initialization scheme for the EM algorithm in `misclassCRC`. It follows the structure of `baffour_misclass_full()` in the accompanying simulation code while generalizing the two-class random split to an arbitrary number of latent classes. The initializer should consume the model matrices and aligned state representation already constructed by `crc_fit()`. Initialization ends when the initial state weights and complete capture-cell counts have been constructed.

2 Notation

For observed unit i , let g denote a candidate configuration of the true values of all misclassified variables and let $k \in \{1, \dots, K\}$ denote a latent class. Write

$$\begin{aligned} M_i(g) &= \Pr(\tilde{G}_i \mid G_i = g), \\ q_{igk}^{(0)} &= \text{initial weight of state } (i, g, k), \\ c(i, g, k) &= \text{complete capture cell associated with state } (i, g, k). \end{aligned}$$

When several variables are misclassified, $M_i(g)$ is the product of their confusion-matrix contributions, consistently with the conditional-independence assumption in the current package specification. When `true_if` establishes a true value, the corresponding contribution is replaced by an indicator that the candidate value equals the observed value.

3 Initial state weights

For each admissible pair (i, g) , draw a mildly perturbed latent-class allocation

$$(a_{ig1}, \dots, a_{igK}) \sim \text{Dirichlet}(\alpha, \dots, \alpha).$$

A moderately large value, such as $\alpha = 20$, keeps the allocation close to $1/K$ but breaks the exact symmetry between latent classes. The initialization should be random whenever $K > 1$; equal deterministic allocation can preserve class symmetry and prevent the latent components from separating. The initial state weights are

$$q_{igk}^{(0)} = \frac{M_i(g)a_{igk}}{\sum_{g'} \sum_{k'=1}^K M_i(g')a_{ig'k'}}.$$

Thus, the weights are non-negative and sum to one within every observed unit. Without misclassification, g has only one admissible value and the formula reduces to the perturbed latent-class allocation. With $K = 1$, every admissible state is weighted only through the misclassification mechanism.

The initializer should use R's current random-number generator state and should not accept a seed argument. A user who requires reproducibility can call `set.seed()` before `crc_fit()` or before the internal initializer is evaluated. This follows the usual R interface and avoids changing the global random-number generator state inside the function.

4 Choice of the Dirichlet concentration

The symmetric Dirichlet concentration α controls the amount of random variation around equal latent-class allocation. Each component has

$$E(a_{igk}) = \frac{1}{K}, \quad \text{Var}(a_{igk}) = \frac{K-1}{K^2(K\alpha+1)}.$$

Values below one favor allocations near the corners of the simplex, whereas large values concentrate the allocations around $1/K$. Initialization needs a moderate concentration: allocations should be unequal enough to break class symmetry but should not routinely create nearly empty classes.

The recommended package default is

$$\alpha = 20.$$

For $K = 2$, the simulation code draws one class proportion uniformly between 0.35 and 0.65. A symmetric beta distribution with the same variance has $\alpha \approx 16.17$, so the proposed default produces a similar, slightly more concentrated random perturbation.

The value should be available as `control$init_alpha`, rather than as a formal argument of `crc_fit()`. The package should require it to be a single, finite, strictly positive number. It is unused when $K = 1$. The package should not calculate α from a target variance: the fixed default keeps the implementation transparent, while the control setting permits deliberate sensitivity analysis. This quantity affects only the random starting values; it is not a parameter of the fitted statistical model.

5 Initial capture-cell counts

For each complete capture cell c , aggregate the state weights as

$$n_c^{(0)} = \sum_i \sum_g \sum_{k=1}^K q_{igk}^{(0)} \mathbf{1}\{c(i, g, k) = c\}.$$

The existing capture-cell index provides the required mapping without a new join. For an all-zero capture cell, there is no observed contribution. A simple initial rule, matching the simulation implementation, is

$$n_c^{(0)} = 1 \quad \text{for every all-zero cell } c.$$

These values are starting values rather than fixed pseudo-observations; the EM updates subsequently replace them with model-based expected counts. This step completes initialization.

6 Recommended initialization sequence

The initializer should perform exactly the following operations:

1. Generate random Dirichlet allocations near equal class probabilities.
2. Multiply them by the applicable misclassification contributions and normalize the resulting state weights within each observed unit.
3. Aggregate state weights into initial expected capture-cell counts.
4. Assign a starting expected count of one to every all-zero capture cell.

The internal initialization object should retain the state weights, expected cell counts, and initialization status. Its dimensions should be checked against the existing state rows and capture-cell rows.

7 Robustness extensions

Because the initialization is random, repeated calls can produce different starting state weights and capture-cell counts. Reproducible collections of starting values can be generated by setting R's random-number seed once before the repeated calls.

8 Points to confirm during implementation

The starting value of one for every all-zero cell and the randomized class split reproduce the essential initialization choices in the simulation code. Before exposing either choice through `control`, they should be checked against the accompanying paper.